

江南大学

课程设计报告

题 目 : *****

学 号 姓 名 1 : 1193230402 甘思源

学 号 姓 名 2 : 1193230401 郑晗湘

学 号 姓 名 3 : 1193230405 郭馨怡

学 号 姓 名 4 : 1193230403 马承芳

课 程 : 虚拟现实

指 导 教 师 : 张怡婕

江南大学

地址：无锡市蠡湖大道 1800 号

二〇二六 年 五 月

目录

第一章 项目概述	1
1.1 项目背景	1
1.2 项目目标	1
1.3 项目概要	1
第二章 项目具体流程和分工	2
2.1 项目分工	2
2.2 项目协作方式与大致流程	3
第三章 三维场景构建	4
3.1 自建模资源	4
3.2 下载建模资源	5
3.2.1 农场模型	5
3.2.2 农作物相关模型及 logo	5
3.2.3 礼物模型及 logo	5
第四章 核心功能实现	7
4.1 甘思源(A)工作	7
4.2 郑晗湘(B)工作	8
4.3 郭馨怡(C)工作	9
4.4 马承芳(D)工作	10
4.4.1 个人负责模块	10
4.4.2 主要脚本工作	10
4.4.3 额外承担的团队工作	12
第五章 实现结果展示	13
5.1 玩家控制	13
5.2 种植系统	14
5.3 牧场系统	15
5.4 经济系统	16
5.5 社交系统	17
5.6 时间与环境	19
5.7 UI 设计	20
第六章 实验心得	21
6.1 甘思源(A)实验心得	21
6.2 郑晗湘(B)实验心得	22
6.3 郭馨怡(C)实验心得	23
6.4 马承芳(D)实验心得	24
附录	25
i. 项目 github 地址	25
ii. 项目完整文件结构	26

第一章 项目概述

1.1 项目背景

随着模拟经营类游戏的发展，农场题材游戏因玩法轻松、目标明确、成长反馈明显，受到许多玩家喜爱。玩家可以在游戏中通过种植、养殖、交易和社交等方式经营自己的农场，体验从生产到收益的完整过程。

本项目基于 Unity 引擎开发，是一款 3D 农场模拟经营游戏。项目围绕农场经营展开，玩家可以在场景中移动，与土地、作物、动物、商店和 NPC 等对象进行交互，完成耕种、收获、售卖、对话和送礼等操作。游戏整体强调简单直观的操作方式，玩家靠近目标后按下交互键即可完成主要行为。

和虚拟现实课程的结合在于，采用第一视角操作，具有虚拟现实的沉浸感；目前使用键鼠操作，后期可再结合真正的虚拟现实设备，实现更具真实感的交互。

1.2 项目目标

本项目的目标是完成一个具有基础可玩性的 3D 农场模拟经营游戏原型。通过项目开发，实现农场经营游戏中常见的核心功能，并形成一個较完整的游戏流程。

具体目标包括：

1. 实现玩家在 3D 场景中的移动、动画和基础交互功能。
2. 实现作物种植系统，包括耕地、播种、生长和收获。
3. 实现动物养殖与产出功能，丰富农场经营内容。
4. 实现背包、金币、商店购买和出货结算等经济系统。
5. 实现 NPC 对话、送礼和好感度变化等社交功能。
6. 实现时间系统，用于驱动物物生长、动物产出、昼夜变化和每日结算。
7. 实现基础 UI 界面，包括时间、金币、背包、商店和对话显示。

1.3 项目概要

本项目采用模块化开发方式，将游戏功能划分为玩家控制、农业种植、牧场养殖、经济系统、NPC 社交、时间管理、环境光照、UI 显示和音效管理等模块。各模块之间通过统一接口和事件机制进行协作，便于小组成员协作。

游戏的核心循环为：玩家种植作物或收集动物产物，将获得的物品存入背包，再通过商店或出货箱进行售卖，获得金币后继续购买种子和道具，从而推动农场持续发展。同时，玩家也可以与 NPC 互动，通过对话和送礼提升好感度，增强游戏的社交体验。

项目整体以 TimeManager 作为时间驱动，作物生长、动物产出、昼夜光照和每日结算等逻辑都围绕游戏时间推进。数据配置方面，项目使用 ScriptableObject 管理作物、物品、动物和 NPC 信息，方便后续扩展和维护。

通过本项目的开发，可以综合运用 Unity 场景搭建、C# 脚本编写、UI 制作、数据配置和模块化设计等知识，最终完成一个结构清晰、功能完整的虚拟现实交互系统。

第二章 项目具体流程和分工

2.1 项目分工

本项目采用模块化分工方式，初期将整体功能划分为四个主要方向，分别由不同成员负责开发。具体分工如表 2-1 所示：

表 2-1 项目分工细则

负责人	功能点	核心负责脚本	开发任务描述
甘思源 (A)	玩家运动交互、牧场系统、房间休息	PlayerController.cs, PlayerInteractor.cs, PlayerAnimator.cs, AnimalEntity.cs, BarnManager.cs, AnimalData.cs, BedInteractable.cs, GameManager.cs, CameraFollow.cs, SceneTransitionManager.cs	负责玩家移动、输入控制、动画同步、相机跟随和统一交互检测。实现牧场系统，包括动物状态、动物产出和动物管理。负责房间休息功能，通过床交互触发跨天逻辑。
郑晗湘 (B)	种地系统	SoilTile.cs, CropEntity.cs, CropData.cs, ParticleManager.cs	负责农业种植核心逻辑，实现土地状态管理，包括耕地、播种、生长、成熟和收获。作物通过时间系统推进生长，并负责耕地、浇水、收获等操作的粒子特效和视觉反馈。
郭馨怡 (C)	商店系统、系统 UI	InventoryManager.cs, ItemData.cs, ShopManager.cs, ShippingBin.cs, UIManager.cs, InventoryUI.cs, ShopUI.cs, TimeUI.cs, PauseMenuUI.cs, MainMenuUI.cs	负责背包、金币、物品数据、商店买卖和出货箱结算。实现各类 UI 面板，包括背包界面、商店界面、时间显示、主菜单和暂停菜单，并处理界面刷新与交互响应。
马承芳 (D)	居民与送礼、昼夜光照、音效制作	TimeManager.cs, ITimeObserver.cs, LightingController.cs, NPCEntity.cs, NPCData.cs, DialogueManager.cs, DialogueUI.cs, AudioManager.cs, EventManager.cs	负责时间管理系统，实现分钟、小时、天数的统一广播。负责昼夜光照变化、NPC 行为逻辑、对话系统、送礼好感度判断，以及背景音乐和交互音效管理。

2.2 项目协作方式与大致流程

每位成员负责自己模块的完整流程，包括脚本编写、数据配置、场景接入和基础测试。各模块之间通过统一的接口和管理器进行连接，避免功能之间直接混乱调用。

项目整体以时间驱动架构为核心，由 **TimeManager** 统一管理游戏时间。作物生长、动物产出、昼夜变化、出货结算和 UI 时间刷新等逻辑都通过时间广播触发。这样可以保证不同模块在同一个时间标准下运行，减少各自单独计时造成的逻辑冲突。

项目的大致开发流程如下：

1. 搭建 Unity 项目基础结构，创建场景、脚本目录和基础资源目录。
2. 完成核心框架，包括玩家控制、交互接口、时间系统、背包系统和 UI 管理器。
3. 各成员根据分工分别开发自己的功能模块。
4. 将农业、牧场、商店、NPC、UI、光照和音效等模块接入主场景。
5. 进行联调测试，检查玩家是否可以完成移动、交互、种植、收获、售卖、对话和送礼等流程。
6. 修复运行过程中的脚本错误、资源引用问题、UI 显示问题和交互逻辑问题。
7. 完成最终整合，形成一个可运行、可演示的 3D 农场模拟经营游戏原型。

在版本协作方面，项目使用 Git 进行管理。部分提交记录如下：



第三章 三维场景构建

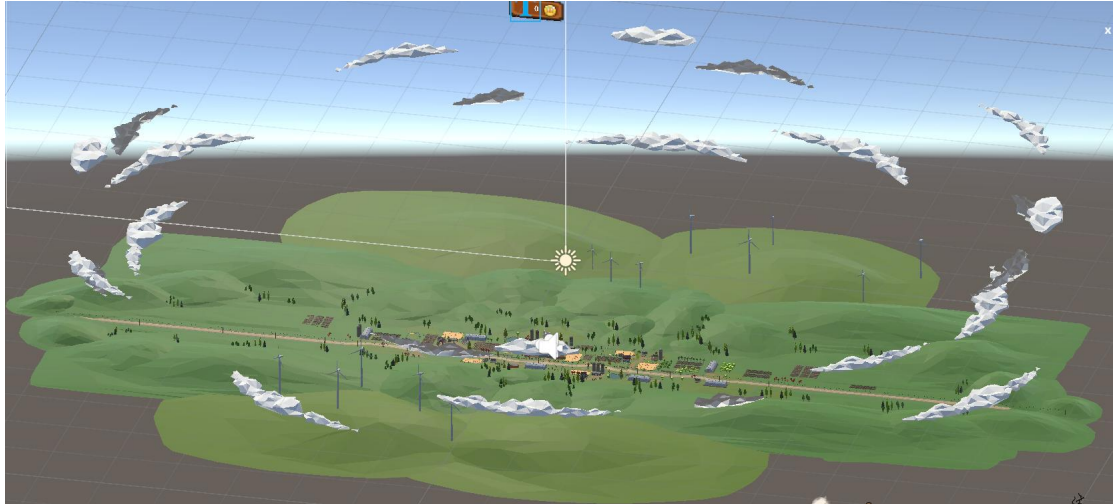
3.1 自建模资源

甘思源:	郑晗湘:
郭馨怡:	马承芳: 为实验 2 中自行建模的模型，在游戏中作为一个可交互的普通 npc 

3.2 下载建模资源

3.2.1 农场模型

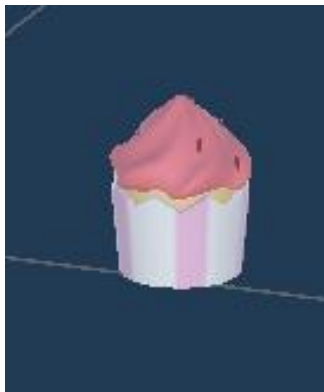
模型来源为 <https://skfb.ly/pGxNU>，模型面积大且空旷，符合农场主题且方便后期加入模型及功能。

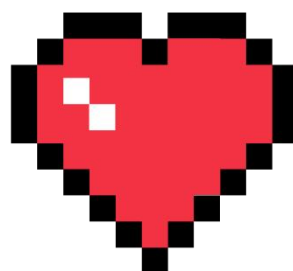


3.2.2 农作物相关模型及 logo

3.2.3 礼物模型及 logo

在网上下载了 3 个小物件模型作为礼物，并且收集了相应的像素风 logo 作为图标。





第四章 核心功能实现

4.1 甘思源(A)工作

4.2 郑晗湘(B)工作

4.3 郭馨怡(C)工作

4.4 马承芳(D)工作

4.4.1 个人负责模块

在个人负责内容方面，我主要承担了 D 模块的开发工作，负责世界环境、NPC 社交、时间系统和音效反馈等内容。该模块虽然不直接负责玩家移动和种植逻辑，但为整个游戏世界的运行提供了基础支撑。例如，作物生长、动物产出、出货结算、昼夜变化等功能都需要依赖统一的时间系统进行驱动；NPC 对话、送礼和商人交互则为游戏增加了社交和经营入口。

在建模与场景部分，我完成了自建模角色“小骑士”的制作，并将其作为项目中的角色资源使用。同时，我参与了主场景 farm 模型在 Unity 中的导入、摆放与碰撞配置工作。为了保证玩家能够在场景中正常行走和交互，我对场景中的主要建筑、道路、装饰物和地形模型进行了基础碰撞设置，使主场景能够满足角色漫游和功能测试的需要。

在交互部分，我主要负责 NPC 对话、送礼、礼物拾取、商人入口、音效播放、时间推进和昼夜变化等功能。玩家靠近 NPC 后可以触发对话；当 NPC 是商人类型时，可以通过交互打开商店界面；玩家拾取场景中的礼物后，可以将其加入背包，并在与 NPC 交互时用于送礼。送礼后系统会根据 NPC 的喜好物品判断是否增加更多好感度，并显示不同的反馈文本。

此外，我还负责环境反馈相关功能，例如根据游戏时间改变太阳角度、光照颜色和环境光强度，使场景呈现从白天到夜晚的变化效果。同时，音效系统会根据不同交互行为播放对应音效，并根据白天或夜晚切换背景音乐，增强游戏的沉浸感。

4.4.2 主要脚本工作

1. TimeManager

TimeManager 是本项目时间系统的核心脚本，负责管理游戏内分钟、小时和天数的推进。该脚本将游戏时间统一封装起来，避免各个模块在自己的脚本中单独计时，从而减少逻辑混乱。

我在该脚本中实现了 OnMinuteChanged、OnHourChanged 和 OnDayPassed 三级时间广播。分钟广播主要用于作物生长、动物产出和 UI 时间刷新；小时广播主要用于昼夜光照变化；天数广播主要用于出货箱结算、跨天刷新和部分状态重置。通过这种方式，项目中所有与时间相关的功能都可以围绕 TimeManager 统一运行。

2. ITimeObserver

ITimeObserver 是时间监听接口，用于规范项目中需要随时间变化的对象接入方式。通过定义统一接口，作物、动物、光照控制器等对象可以按照统一规则监听时间变化。

该接口的作用是降低脚本之间的耦合度，使各模块不需要直接依赖彼此，只需要接入时间系统即可完成更新逻辑。这种设计也方便后期扩展新的时间驱动功能，例如天气变化、NPC 日程或节日事件等。

3. EventManager

EventManager 是全局事件总线，用于处理跨模块通信。项目中存在一些不适合直接调用的功能，例如睡觉跨天、出货结算、UI 刷新和系统状态变化等。如果脚本之间相互直接引用，会导致代码耦合度过高，后期维护困难。

因此，我通过 **EventManager** 将部分跨模块行为封装为事件。其他系统只需要监听对应事件，在事件触发时执行自己的逻辑即可。这样可以让各模块保持独立，同时又能完成必要的的数据同步和流程联动。

4. **LightingController**

LightingController 负责根据游戏时间更新场景光照，实现昼夜变化效果。该脚本会监听 **TimeManager** 中的小时变化，并根据当前时间调整太阳旋转角度、方向光颜色、环境光颜色和光照强度。

在白天时，场景光照较亮，颜色偏自然；到傍晚和夜晚时，光照逐渐变暗，环境颜色也随之发生变化。该功能让场景不再是固定光照状态，而是能够随着时间流动产生变化，提高了农场场景的真实感和沉浸感。

5. **NPCEntity、NPCData、DialogueManager、DialogueUI**

NPC 系统是我负责的主要交互模块之一。其中，**NPCData** 使用 **ScriptableObject** 保存 NPC 的基础数据，包括 NPC 名称、头像、默认对话内容、喜好物品、送礼反馈文本和好感度数值等。这样可以在 Unity 编辑器中直接配置不同 NPC 的信息，方便后续扩展更多角色。

NPCEntity 负责 NPC 在场景中的交互入口。当玩家靠近 NPC 并触发交互时，该脚本会判断当前 NPC 的类型和状态。如果是普通 NPC，则进入对话流程；如果是商人 NPC，则可以进一步打开商店界面。

DialogueManager 负责管理对话流程，包括选择 NPC 对话内容、处理送礼逻辑、判断玩家送出的物品是否符合 NPC 喜好，以及修改 NPC 好感度。**DialogueUI** 则负责将对话内容显示到界面上，包括 NPC 名称、头像和文本内容等。通过这几个脚本配合，实现了 NPC 对话、送礼、好感度反馈和商人入口等功能。

6. **GiftPickup**

GiftPickup 用于实现场景中礼物道具的拾取功能。玩家靠近礼物并触发交互后，系统会将对应礼物数据加入背包，同时移除或隐藏场景中的礼物对象。

该功能与背包系统和 NPC 送礼系统形成连接。玩家先从场景中拾取礼物，再将礼物送给 NPC，从而触发好感度变化。它使 NPC 社交系统不只是单独的对话功能，而是和场景探索、物品收集产生了关联。

7. **AudioManager**

AudioManager 负责统一管理项目中的音效播放和背景音乐切换。我在该脚本中实现了统一的音效播放接口，使其他模块在需要播放音效时不需要重复编写播放逻辑，只需要调用对应方法即可。

同时，**AudioManager** 会根据游戏时间切换白天和夜晚的背景音乐，使声音表现与游戏环境变化保持一致。例如白天播放较轻松的农场背景音乐，夜晚则切换为更安静的音乐氛围。该功能增强了游戏反馈，也让玩家更容易感受到时间变化。

8. **BatchAddConvexColliders**

`BatchAddConvexColliders` 是我额外编写的 Unity 编辑器工具，主要用于提高主场景碰撞配置效率。由于主场景模型包含较多子网格，如果手动为每个模型添加碰撞体，会耗费大量时间，并且容易遗漏。

因此，我编写该工具用于批量为主场景模型及其子网格添加 `Convex MeshCollider`。通过该工具，可以快速完成场景中大批量模型的基础碰撞配置，使玩家角色能够在场景中正常移动，并避免穿过建筑、装饰物或地面模型。该工具提高了场景搭建效率，也减少了重复性手动操作。

4.4.3 额外承担的团队工作

除 D 模块本职开发外，我还承担了部分项目组织和工程管理工作，包括项目总体文件夹架构设计、团队任务分工、开发规划、GitHub 仓库建立以及协作规范整理等内容。

在项目初期，我负责 Unity 工程的整体目录结构，将资源按照功能类型进行分类管理，例如场景、脚本、模型、材质、贴图、音频、预制体和数据文件等。脚本部分也按照功能模块进行划分，包括 `Core`、`Player`、`Farming`、`Economy`、`Social`、`UI`、`Environment`、`Audio` 等目录。这样的目录结构可以帮助团队成员快速找到自己负责的文件，也能减少多人协作时资源混乱的问题。

在团队分工方面，我负责项目功能拆分和任务安排。根据项目需求，将整体功能划分为玩家控制、农业种植、经济 UI、世界社交四个主要方向，并明确每个成员负责的脚本和功能边界。通过这种方式，每位成员都能围绕自己的模块进行开发，最后再通过统一接口和公共管理器进行整合。

在 GitHub 协作方面，我负责建立项目仓库，并整理了基本协作规范。我在协作规范中强调 `.meta` 文件需要和对应资源一起提交，场景和预制体尽量避免多人同时修改，提交信息需要尽量清晰。

我还参与了主场景 `farm` 的搭建与整合工作，包括导入农场场景模型、调整模型位置、配置基础碰撞体，以及检查玩家在主场景中的移动和交互测试效果。

总体来说，我负责的工作不仅包括 NPC、时间、光照和音效等具体功能开发，也包括部分场景配置、工具编写和团队工程管理工作。这些内容共同保证了项目能够从单个功能脚本逐步整合为一个完整的 3D 农场模拟虚拟场景。

另外，在开发结束后，由我进行报告的整合和大组报告撰写。

第五章 实现结果展示

本项目最终完成了一个基于 Unity 的 3D 农场模拟经营游戏原型。游戏能够在主农场场景中运行，玩家可以控制角色进行移动，并与场景中的土地、作物、动物、NPC、商店和出货箱等对象进行交互。整体功能围绕农场经营展开，形成了较完整的生产、收集、交易和社交流程。

5.1 玩家控制

在玩家控制方面，项目实现了

5.2 种植系统

在种植系统方面，项目实现了

5.3 牧场系统

在牧场系统方面，项目实现了

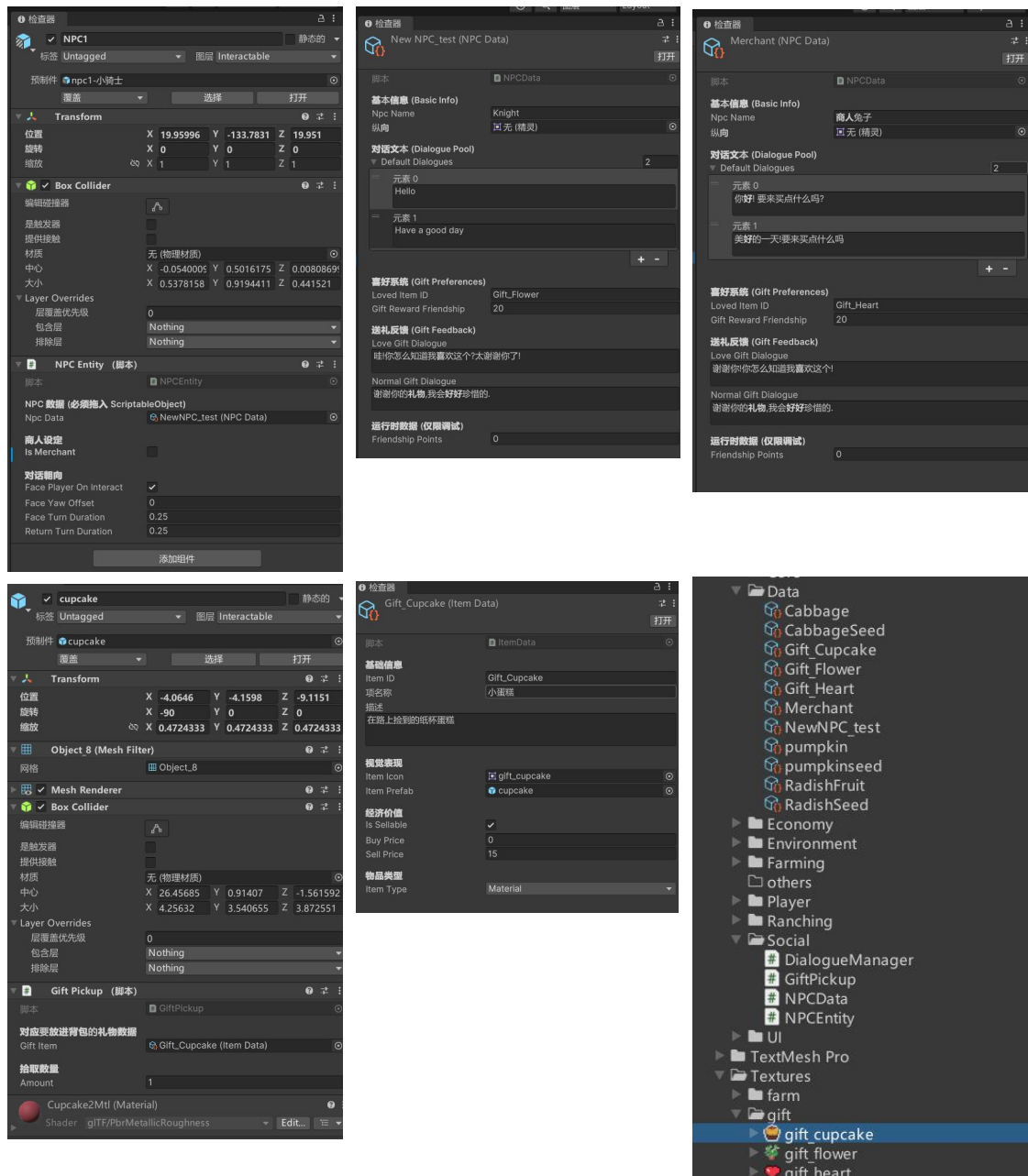
5.4 经济系统

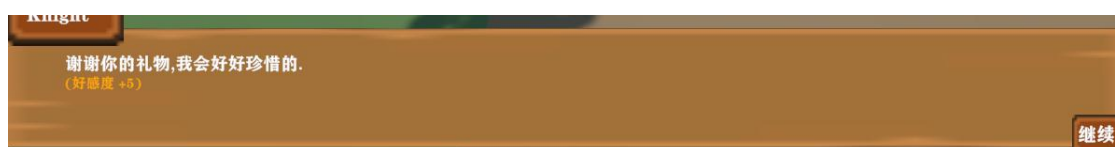
在经济系统方面，项目实现了

5.5 社交系统

在 NPC 社交系统方面，项目实现了 NPC 数据配置、对话显示、送礼、好感度变化和商人入口功能。玩家可以与 NPC 进行对话，也可以赠送礼物提升好感度。当 NPC 为商人角色时，还可以通过交互进入商店界面，使 NPC 系统与经济系统产生关联。

实现截图如下：

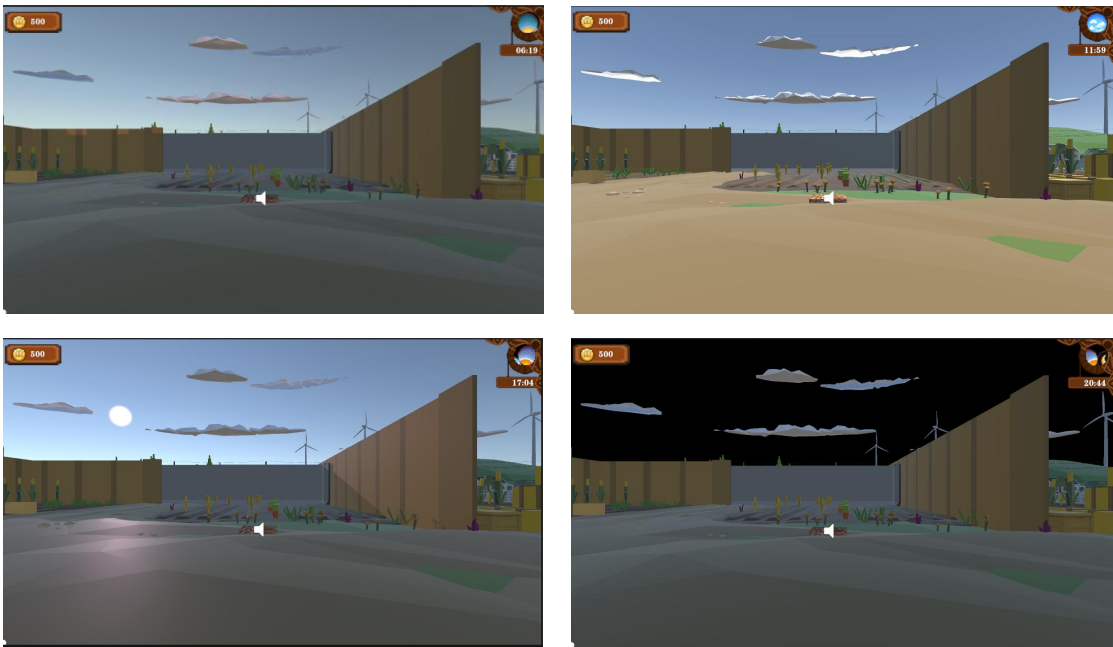
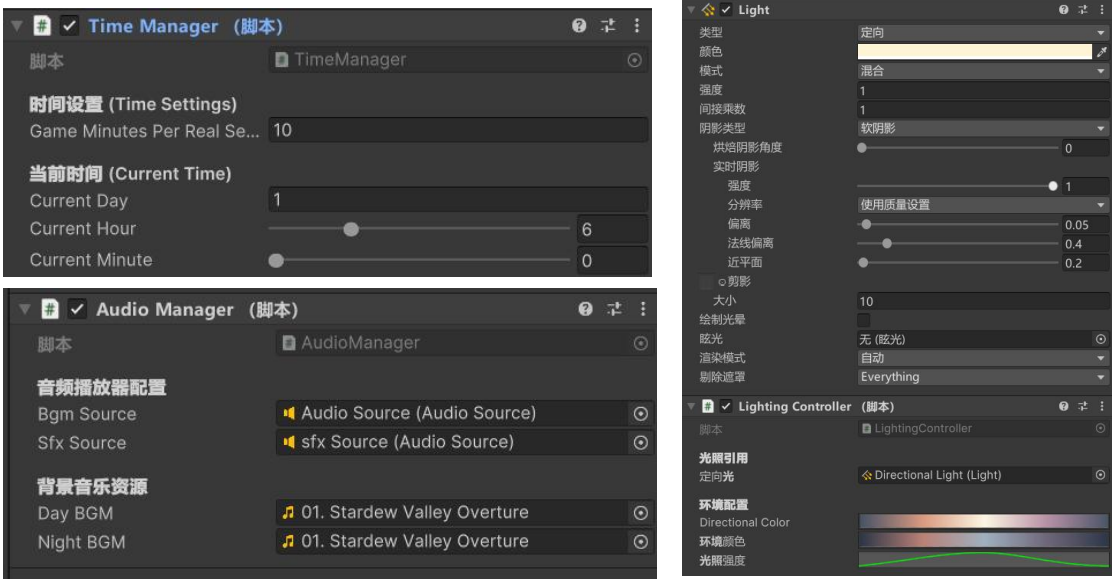




5.6 时间与环境

在时间与环境系统方面，项目实现了游戏内分钟、小时和天数的推进，并通过时间广播驱动作物生长、动物产出、出货结算、UI 时间刷新和昼夜光照变化。场景光照会根据游戏时间发生变化，使农场环境具有昼夜交替效果。

同时，项目加入了背景音乐和部分交互音效，使玩家在操作时能够获得更明显的反馈。



5.7 UI 设计

在 UI 和音效方面，项目实现了时间显示、金币显示、背包界面、商店界面、对话界面、主菜单和暂停菜单等基础界面。

第六章 实验心得

6.1 甘思源(A)实验心得

6.2 郑晗湘(B)实验心得

6.3 郭馨怡(C)实验心得

6.4 马承芳(D)实验心得

通过本次 3D 农场模拟经营项目的开发，我对 Unity 项目的整体开发流程有了更加系统的认识。相比单独完成一个小功能，本项目涉及玩家交互、时间推进、NPC 社交、商店、种植、音效、光照和 UI 等多个模块，各模块之间既相互独立，又需要在最终场景中配合运行。因此，在开发过程中，我不仅熟悉了具体脚本的编写方法，也更加理解了项目结构设计和团队协作的重要性。

在技术实现方面，我熟悉了 Unity 项目中脚本单例、事件广播、ScriptableObject 数据配置、UI 按钮绑定、音频播放、光照变化和碰撞体配置等流程。例如，在实现 TimeManager 时，我认识到统一时间管理对于模拟经营类游戏非常重要。如果每个系统都使用自己的计时逻辑，后期很容易出现作物生长、动物产出、光照变化和每日结算不同步的问题。因此，我通过分钟、小时、天数三级广播的方式，让其他模块根据统一时间事件执行各自逻辑，提高了系统的稳定性和可维护性。

在 NPC 和对话系统开发中，我进一步理解了数据驱动设计的优势。通过使用 ScriptableObject 配置 NPC 的名字、头像、对话文本、喜好物品和好感度奖励，可以减少代码中的硬编码内容，也方便在 Unity 编辑器中直接调整角色数据。这样如果后续需要增加新的 NPC 或修改 NPC 对话内容，不需要频繁改动脚本，只需要创建或修改对应的数据资源即可。

在 UI 和交互方面，我学习了如何将脚本逻辑与 Unity 界面组件进行绑定。例如 NPC 对话框需要显示角色名称、头像和对话内容，送礼按钮需要与背包数据、NPC 喜好和好感度变化联动。这个过程让我认识到 UI 并不只是显示信息，还需要承担玩家操作入口和状态反馈的作用。只有当交互逻辑和界面反馈配合好时，玩家才能清楚知道自己当前完成了什么操作。

在团队协作方面，本次项目让我更加理解文件结构、版本管理和模块边界的重要性。Unity 项目资源较多，包括脚本、场景、预制体、材质、贴图、音频和 .meta 文件，如果没有清晰的目录结构，很容易出现资源混乱和引用丢失的问题。因此，在项目中按模块划分文件夹，可以让成员更清楚地管理自己负责的内容。使用 GitHub 协作时，也需要注意及时提交 .meta 文件，避免多人同时修改同一个场景或预制体，减少合并冲突。

通过本次实验，我不仅完成了自己负责的 D 模块功能，也参与了项目结构整理、场景整合和协作规范制定。整个过程让我认识到，一个完整游戏项目不仅需要单个功能能够运行，还需要各模块之间能够稳定衔接。今后如果继续完善该项目，我希望进一步优化 NPC 行为逻辑、增加更多对话内容和事件触发机制，并完善存档、任务和天气等系统，使游戏体验更加完整。

附录

i. 项目 github 地址

<https://github.com/nekoneko2333/26-Spring-VR-3DFarm>

项目详情：

The screenshot shows the GitHub repository page for '26-Spring-VR-3DFarm'. The repository is public and has 39 commits. The main branch is 'main'. The repository contains several files and folders, including .vscode, Assets, Packages, ProjectSettings, .gitattributes, .gitignore, .vsconfig, C.cmd, D.cmd, README.md, git, main, 实验3小组报告模板.docx, and 实验3小组报告模板.pdf. The right sidebar shows the repository's metadata, including the README, Activity, Stars, Watching, and Forks. The Contributors section lists three contributors: nekoneko2333, labyrinth-07, and xin-huan. The Languages section shows the repository is primarily composed of ShaderLab (45.5%) and C# (41.9%).

File/Folder	Description	Commit Time
.vscode	C金钱系统及对接文档	2 weeks ago
Assets	Merge branch 'main' of https://github.com/nekoneko2333/2...	3 hours ago
Packages	金币and时间UI	2 weeks ago
ProjectSettings	做好了MainMenu，对话UI已经对接好O做的了，商店打开的...	2 weeks ago
.gitattributes	feat: 修改gitattribute文件	3 weeks ago
.gitignore	feat: 项目骨架初始化，包含时间驱动解构规范	3 weeks ago
.vsconfig	feat: 完善农场土地状态、多作物收割及死亡逻辑	2 weeks ago
C.cmd	做好了MainMenu，对话UI已经对接好O做的了，商店打开的...	2 weeks ago
D.cmd	[O]完善了礼物data以及抽取等逻辑，更新对接文档	2 weeks ago
README.md	提交送礼有关脚本的修改，以及对接文档更新	2 weeks ago
git	保存我写的代码	2 weeks ago
main	保存我自己修改的部分	2 days ago
实验3小组报告模板.docx	[O]优化脚本，写了个总实验报告初稿	3 hours ago
实验3小组报告模板.pdf	[O]优化脚本，写了个总实验报告初稿	3 hours ago

ii. 项目完整文件结构

```
Assets/
├─ Animations/           # 玩家、动物、NPC 动画状态机
├─ Audio/                # BGM、环境音、UI/交互音效 (D)
├─ Materials/            # 场景、角色、作物所有材质
├─ Models/
│   ├─ Characters/       # 玩家、NPC 模型
│   ├─ Crops/            # 作物各阶段模型 (B)
│   ├─ Environment/      # 房屋、农场、装饰物
│   └─ Objects/          # 种子、农具、掉落物模型
├─ Prefabs/
│   ├─ Actors/           # 玩家、NPC、动物预制体
│   ├─ Interactable_objects/ # 土地、出货箱、商店 NPC 预制体
│   └─ UI/               # 各类 UI 面板预制体 (C)
├─ Scenes/
│   ├─ farm.unity        # 主农场场景
│   ├─ home.unity        # 室内场景 (A)
│   └─ shop.unity        # 商店/城镇场景 (C)
├─ Scripts/
│   ├─ Core/
│   │   ├─ GameManager.cs # 游戏主状态控制 (A)
│   │   ├─ TimeManager.cs # 时间广播中心 (D)
│   │   ├─ EventManager.cs # 全局事件总线 (D)
│   │   ├─ ITimeObserver.cs # 时间监听接口 (D)
│   │   ├─ SceneTransitionManager.cs # 场景跳转 (A)
│   │   ├─ CameraFollow.cs # 相机平滑控制 (A)
│   │   └─ IInteractable.cs # 交互通用接口 (A)
│   ├─ Player/
│   │   ├─ PlayerController.cs # 移动与输入 (A)
│   │   ├─ PlayerInteractor.cs # 射线检测交互中心 (A)
│   │   └─ PlayerAnimator.cs # 动画参数同步 (A)
│   ├─ Farming/
│   │   ├─ SoilTile.cs     # 土地逻辑 (B)
│   │   ├─ CropEntity.cs   # 作物生长逻辑 (B)
│   │   ├─ CropData.cs     # 作物 SO 配置 (B)
│   │   ├─ ShippingBin.cs  # 出货箱结算 (C)
│   │   └─ ParticleManager.cs # 特效管理 (B)
│   ├─ Economy/
│   │   ├─ InventoryManager.cs # 物品/金币数据库 (C)
│   │   ├─ ItemData.cs      # 物品 SO 配置 (C)
│   │   └─ ShopManager.cs   # 商店买卖系统 (C)
│   └─ UI/
```

- | | └─ UIManager.cs # UI 栈管理 (C)
- | | └─ TimeUI.cs # 时间显示更新 (C)
- | | └─ InventoryUI.cs # 背包界面控制 (C)
- | | └─ ShopUI.cs # 商店界面控制 (C)
- | | └─ DialogueUI.cs # 对话框表现 (D)
- | | └─ MainMenuUI.cs # 主菜单逻辑 (C)
- | | └─ PauseMenuUI.cs # 暂停菜单逻辑 (C)
- | └─ Social/
- | | └─ NPCEntity.cs # NPC 交互逻辑 (D)
- | | └─ NPCData.cs # NPC 属性 SO 配置 (D)
- | | └─ DialogueManager.cs # 对话文本调度 (D)
- | └─ Ranching/
- | | └─ BarnManager.cs # 动物管理中心 (A)
- | | └─ AnimalEntity.cs # 动物 AI 与产出 (A)
- | | └─ AnimalData.cs # 动物 SO 配置 (A)
- | └─ Environment/
- | | └─ LightingController.cs # 昼夜光照控制 (D)
- | | └─ WeatherManager.cs # 简单天气控制 (D)
- | | └─ BedInteractable.cs # 睡觉跨天触发器 (A)
- | └─ Audio/
- | | └─ AudioManager.cs # 音量与播放控制 (D)
- └─ Settings/ # URP 渲染设置与 InputActions
- └─ Textures/ # 所有的贴图、UI 图标